
TEORÍA DE BASE DE DATOS PROCESAMIENTO DE CONSULTAS

Prof. Sergio Salinas

TRANSFORMACIÓN DE EXPRESIONES RELACIONALES

1 Configuración entorno de trabajo.

```
CREATE DATABASE testuni;
```

1.1 Tabla department

```
CREATE TABLE department (  
    dept_name VARCHAR(50) PRIMARY KEY,  
    building VARCHAR(50),  
    budget NUMERIC(12,2)  
);
```

1.2 Tabla instructor

```
CREATE TABLE instructor (  
    ID INTEGER PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    dept_name VARCHAR(50),  
    salary NUMERIC(10,2),  
    CONSTRAINT fk_instructor_department  
        FOREIGN KEY (dept_name)  
        REFERENCES department(dept_name)  
);
```

1.3 Tabla course

```
CREATE TABLE course (  
    course_id VARCHAR(20) PRIMARY KEY,  
    title VARCHAR(100) NOT NULL,  
    dept_name VARCHAR(50),  
    credits INTEGER,  
    CONSTRAINT fk_course_department  
        FOREIGN KEY (dept_name)  
        REFERENCES department(dept_name)  
);
```

1.4 Tabla teaches

```
CREATE TABLE teaches (  
    ID INTEGER,  
    course_id VARCHAR(20),  
    sec_id VARCHAR(10),
```

```
semester VARCHAR(20),
year INTEGER,
PRIMARY KEY (ID, course_id, sec_id, semester, year),
CONSTRAINT fk_teaches_instructor
    FOREIGN KEY (ID)
    REFERENCES instructor(ID),
CONSTRAINT fk_teaches_course
    FOREIGN KEY (course_id)
    REFERENCES course(course_id)
);
```

2 Inserción de registros de prueba

2.1 Registros para department

```
INSERT INTO department (dept_name, building, budget) VALUES
('Music', 'Mozart Hall', 120000.00),
('History', 'Taylor Hall', 90000.00),
('Comp. Sci.', 'Watson Hall', 250000.00),
('Physics', 'Newton Hall', 180000.00),
('Biology', 'Darwin Hall', 160000.00);
```

2.2 Registros para instructor

```
INSERT INTO instructor (ID, name, dept_name, salary) VALUES
(10101, 'Srinivasan', 'Comp. Sci.', 65000.00),
(12121, 'Wu', 'Music', 90000.00),
(15151, 'Mozart', 'Music', 40000.00),
(22222, 'Einstein', 'Physics', 95000.00),
(32343, 'El Said', 'History', 60000.00),
(33456, 'Gold', 'Physics', 87000.00),
(45565, 'Katz', 'Comp. Sci.', 75000.00),
(58583, 'Califieri', 'History', 62000.00),
(76543, 'Singh', 'Biology', 72000.00),
(98345, 'Kim', 'Music', 68000.00);
```

2.3 Registros para course

```
INSERT INTO course (course_id, title, dept_name, credits) VALUES
('CS-101', 'Intro. to Computer Science', 'Comp. Sci.', 4),
('CS-347', 'Database System Concepts', 'Comp. Sci.', 4),
('MUS-101', 'Introduction to Music', 'Music', 3),
('MUS-201', 'Music Theory', 'Music', 4),
('HIS-351', 'World History', 'History', 3),
('PHY-101', 'Physics I', 'Physics', 4),
('PHY-202', 'Quantum Mechanics', 'Physics', 4),
('BIO-101', 'Introduction to Biology', 'Biology', 3),
('BIO-301', 'Genetics', 'Biology', 4);
```

2.4 Registros para teaches

```
INSERT INTO teaches (ID, course_id, sec_id, semester, year) VALUES
```

```
(10101, 'CS-101', '1', 'Fall', 2017),  
(10101, 'CS-347', '1', 'Spring', 2018),  
(45565, 'CS-101', '2', 'Fall', 2017),  
(45565, 'CS-347', '1', 'Fall', 2019),  
(12121, 'MUS-101', '1', 'Fall', 2017),  
(12121, 'MUS-201', '1', 'Spring', 2018),  
(15151, 'MUS-101', '2', 'Fall', 2017),  
(98345, 'MUS-201', '2', 'Fall', 2019),  
(32343, 'HIS-351', '1', 'Spring', 2017),  
(58583, 'HIS-351', '2', 'Fall', 2018),  
(22222, 'PHY-101', '1', 'Fall', 2017),  
(33456, 'PHY-202', '1', 'Spring', 2018),  
(76543, 'BIO-101', '1', 'Fall', 2017),  
(76543, 'BIO-301', '1', 'Spring', 2019);
```

3 Reglas de equivalencia del álgebra relacional y sus ejemplos en SQL

Nº	Regla de equivalencia	Ejemplo SQL equivalente
1	$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$	Forma 1: <pre>SELECT * FROM instructor WHERE dept_name = 'Music' AND salary > 70000;</pre> Forma 2: <pre>SELECT * FROM (SELECT * FROM instructor WHERE salary > 70000) AS t WHERE dept_name = 'Music';</pre>
2	$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$	Forma 1: <pre>SELECT * FROM (SELECT * FROM instructor WHERE salary > 70000) AS t WHERE dept_name = 'Music';</pre> Forma 2: <pre>SELECT * FROM (SELECT * FROM instructor WHERE dept_name = 'Music') AS t WHERE salary > 70000;</pre>
3	$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) \equiv \Pi_{L_1}(E), \text{ si } L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$	Forma 1: <pre>SELECT name FROM (SELECT ID, name, dept_name FROM instructor) AS t;</pre> Forma 2: <pre>SELECT name FROM instructor;</pre>
4.a	$\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$	Forma 1: <pre>SELECT * FROM instructor i CROSS JOIN teaches t WHERE i.ID = t.ID;</pre> Forma 2: <pre>SELECT * FROM instructor i JOIN teaches t ON i.ID = t.ID;</pre>

Nº	Regla de equivalencia	Ejemplo SQL equivalente
4.b	$\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$	Forma 1: <pre>SELECT * FROM instructor i JOIN teaches t ON i.ID = t.ID WHERE t.year = 2017;</pre> Forma 2: <pre>SELECT * FROM instructor i JOIN teaches t ON i.ID = t.ID AND t.year = 2017;</pre>
5	$E_1 \bowtie_{\theta} E_2 \equiv E_2 \bowtie_{\theta} E_1$	Forma 1: <pre>SELECT * FROM instructor i JOIN teaches t ON i.ID = t.ID;</pre> Forma 2: <pre>SELECT * FROM teaches t JOIN instructor i ON i.ID = t.ID;</pre>
6.a	$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$	Forma 1: <pre>SELECT * FROM (instructor i JOIN teaches t ON i.ID = t.ID) JOIN course c ON t.course_id = c.course_id;</pre> Forma 2: <pre>SELECT * FROM instructor i JOIN (teaches t JOIN course c ON t.course_id = c.course_id) ON i.ID = t.ID;</pre>
6.b	$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$	Forma 1: <pre>SELECT * FROM (instructor i JOIN teaches t ON i.ID = t.ID) JOIN course c ON t.course_id = c.course_id AND c.credits > 3;</pre> Forma 2: <pre>SELECT * FROM instructor i JOIN (teaches t JOIN course c ON t.course_id = c.course_id) ON i.ID = t.ID AND c.credits > 3;</pre>

Nº	Regla de equivalencia	Ejemplo SQL equivalente
7.a	$\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} E_2$, si θ_1 usa solo atributos de E_1	Forma 1: <pre>SELECT * FROM instructor i JOIN teaches t ON i.ID = t.ID WHERE i.dept_name = 'Music';</pre> Forma 2: <pre>SELECT * FROM (SELECT * FROM instructor WHERE dept_name = 'Music') i JOIN teaches t ON i.ID = t.ID;</pre>
7.b	$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$	Forma 1: <pre>SELECT * FROM instructor i JOIN teaches t ON i.ID = t.ID WHERE i.dept_name = 'Music' AND t.year = 2017;</pre> Forma 2: <pre>SELECT * FROM (SELECT * FROM instructor WHERE dept_name = 'Music') i JOIN (SELECT * FROM teaches WHERE year = 2017) t ON i.ID = t.ID;</pre>
8.a	$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$, si θ usa atributos de $L_1 \cup L_2$	Forma 1: <pre>SELECT i.ID, t.course_id FROM instructor i JOIN teaches t ON i.ID = t.ID;</pre> Forma 2: <pre>SELECT i.ID, t.course_id FROM (SELECT ID FROM instructor) i JOIN (SELECT ID, course_id FROM teaches) t ON i.ID = t.ID;</pre>

Nº	Regla de equivalencia	Ejemplo SQL equivalente
8.b	$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$	Forma 1: <pre>SELECT i.name, t.course_id FROM instructor i JOIN teaches t ON i.ID = t.ID;</pre> Forma 2: <pre>SELECT i.name, t.course_id FROM (SELECT ID, name FROM instructor) i JOIN (SELECT ID, course_id FROM teaches) t ON i.ID = t.ID;</pre>
9.a	$E_1 \cup E_2 \equiv E_2 \cup E_1$	Forma 1: <pre>SELECT course_id FROM teaches WHERE semester = 'Fall' UNION SELECT course_id FROM teaches WHERE semester = 'Spring';</pre> Forma 2: <pre>SELECT course_id FROM teaches WHERE semester = 'Spring' UNION SELECT course_id FROM teaches WHERE semester = 'Fall';</pre>
9.b	$E_1 \cap E_2 \equiv E_2 \cap E_1$	Forma 1: <pre>SELECT course_id FROM teaches WHERE semester = 'Fall' INTERSECT SELECT course_id FROM teaches WHERE year = 2017;</pre> Forma 2: <pre>SELECT course_id FROM teaches WHERE year = 2017 INTERSECT SELECT course_id FROM teaches WHERE semester = 'Fall';</pre>

Nº	Regla de equivalencia	Ejemplo SQL equivalente
10.a	$(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$	Forma 1: <pre>(SELECT course_id FROM teaches WHERE semester = 'Fall' UNION SELECT course_id FROM teaches WHERE semester = 'Spring') UNION SELECT course_id FROM teaches WHERE semester = 'Summer';</pre> Forma 2: <pre>SELECT course_id FROM teaches WHERE semester = 'Fall' UNION (SELECT course_id FROM teaches WHERE semester = 'Spring' UNION SELECT course_id FROM teaches WHERE semester = 'Summer');</pre>
10.b	$(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$	Forma 1: <pre>(SELECT course_id FROM teaches WHERE semester = 'Fall' INTERSECT SELECT course_id FROM teaches WHERE year = 2017) INTERSECT SELECT course_id FROM teaches WHERE sec_id = '1';</pre> Forma 2: <pre>SELECT course_id FROM teaches WHERE semester = 'Fall' INTERSECT (SELECT course_id FROM teaches WHERE year = 2017 INTERSECT SELECT course_id FROM teaches WHERE sec_id = '1');</pre>

Nº	Regla de equivalencia	Ejemplo SQL equivalente
11	a. $\sigma_{\theta}(E_1 \cup E_2) \equiv \sigma_{\theta}(E_1) \cup \sigma_{\theta}(E_2)$ b. $\sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap \sigma_{\theta}(E_2)$ c. $\sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$ d. $\sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap E_2$ e. $\sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - E_2$	Con unión – Forma 1: <pre>SELECT * FROM (SELECT * FROM course WHERE dept_name = 'Music' UNION SELECT * FROM course WHERE dept_name = 'History') AS q WHERE credits > 3;</pre> Con unión – Forma 2: <pre>SELECT * FROM course WHERE dept_name = 'Music' AND credits > 3 UNION SELECT * FROM course WHERE dept_name = 'History' AND credits > 3;</pre>
12	$\Pi_L(E_1 \cup E_2) \equiv \Pi_L(E_1) \cup \Pi_L(E_2)$, si E_1 y E_2 tienen el mismo esquema	Forma 1: <pre>SELECT course_id FROM (SELECT * FROM course WHERE dept_name = 'Music' UNION SELECT * FROM course WHERE dept_name = 'History') AS q;</pre> Forma 2: <pre>SELECT course_id FROM course WHERE dept_name = 'Music' UNION SELECT course_id FROM course WHERE dept_name = 'History';</pre>
13	$\sigma_{\theta}(G\gamma_A(E)) \equiv G\gamma_A(\sigma_{\theta}(E))$, si θ usa solo atributos de agrupamiento G	Forma 1: <pre>SELECT * FROM (SELECT dept_name, COUNT(*) AS total FROM instructor GROUP BY dept_name) AS g WHERE dept_name = 'Music';</pre> Forma 2: <pre>SELECT dept_name, COUNT(*) AS total FROM instructor WHERE dept_name = 'Music' GROUP BY dept_name;</pre>

Nº	Regla de equivalencia	Ejemplo SQL equivalente
14.a	$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$	Forma 1: <pre>SELECT * FROM instructor i FULL OUTER JOIN teaches t ON i.ID = t.ID;</pre> Forma 2: <pre>SELECT * FROM teaches t FULL OUTER JOIN instructor i ON i.ID = t.ID;</pre>
14.b	$E_1 \ltimes E_2 \equiv E_2 \ltimes E_1$	Forma 1: <pre>SELECT * FROM instructor i LEFT JOIN teaches t ON i.ID = t.ID;</pre> Forma 2: <pre>SELECT * FROM teaches t RIGHT JOIN instructor i ON i.ID = t.ID;</pre>
15.a	$\sigma_{\theta_1}(E_1 \ltimes_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \ltimes_{\theta} E_2$, si θ_1 usa solo atributos de E_1	Forma 1: <pre>SELECT * FROM instructor i LEFT JOIN teaches t ON i.ID = t.ID WHERE i.dept_name = 'Music';</pre> Forma 2: <pre>SELECT * FROM (SELECT * FROM instructor WHERE dept_name = 'Music') i LEFT JOIN teaches t ON i.ID = t.ID;</pre>
15.b	$\sigma_{\theta_1}(E_2 \ltimes_{\theta} E_1) \equiv E_2 \ltimes_{\theta} (\sigma_{\theta_1}(E_1))$, si θ_1 usa solo atributos de E_1	Forma 1: <pre>SELECT * FROM teaches t RIGHT JOIN instructor i ON i.ID = t.ID WHERE i.dept_name = 'Music';</pre> Forma 2: <pre>SELECT * FROM teaches t RIGHT JOIN (SELECT * FROM instructor WHERE dept_name = 'Music') i ON i.ID = t.ID;</pre>

Nº	Regla de equivalencia	Ejemplo SQL equivalente
16.a	$\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2)$, si θ_1 rechaza nulos de E_2	Forma 1: <pre>SELECT * FROM instructor i LEFT JOIN teaches t ON i.ID = t.ID WHERE t.year = 2017;</pre> Forma 2: <pre>SELECT * FROM instructor i JOIN teaches t ON i.ID = t.ID WHERE t.year = 2017;</pre>
16.b	$\sigma_{\theta_1}(E_2 \bowtie_{\theta} E_1) \equiv \sigma_{\theta_1}(E_2 \bowtie_{\theta} E_1)$, si θ_1 rechaza nulos del lado externo	Forma 1: <pre>SELECT * FROM teaches t RIGHT JOIN instructor i ON i.ID = t.ID WHERE t.year = 2017;</pre> Forma 2: <pre>SELECT * FROM teaches t JOIN instructor i ON i.ID = t.ID WHERE t.year = 2017;</pre>

4 Ejemplo 1: optimización de una consulta

Supongamos la siguiente consulta SQL:

```
SELECT i.name, c.title
FROM instructor i
JOIN teaches t
  ON i.ID = t.ID
JOIN course c
  ON t.course_id = c.course_id
WHERE i.dept_name = 'Music';
```

La consulta busca los nombres de los instructores del departamento **Music** y los títulos de los cursos que enseñan.

5 Expresión en álgebra relacional

La consulta puede expresarse en álgebra relacional de la siguiente manera:

$$\Pi_{name, title} (\sigma_{dept_name='Music'} (instructor \bowtie teaches \bowtie course))$$

Donde:

- $\Pi_{name, title}$ representa la proyección de los atributos **name** y **title**.
- $\sigma_{dept_name='Music'}$ representa la selección de los instructores del departamento **Music**.
- $\bowtie$ representa las operaciones de join entre las relaciones.

6 Planes equivalentes

El optimizador puede generar varios planes equivalentes. Aunque todos producen el mismo resultado, pueden tener costos muy diferentes.

6.1 Plan A: hacer primero los joins y luego filtrar

Una primera alternativa consiste en realizar primero los joins y aplicar después la selección.

$$\Pi_{name, title} (\sigma_{dept_name='Music'} ((instructor \bowtie teaches) \bowtie course))$$

Este plan puede describirse así:

1. Unir **instructor** con **teaches**.
2. Unir el resultado con **course**.
3. Filtrar las tuplas donde **dept_name = 'Music'**.
4. Proyectar los atributos **name** y **title**.

El problema de este plan es que puede generar resultados intermedios grandes antes de aplicar el filtro.

6.2 Plan B: filtrar primero y luego hacer los joins

Una segunda alternativa consiste en aplicar primero la selección sobre **instructor** y luego hacer los joins.

$$\Pi_{name, title} ((\sigma_{dept_name='Music'}(instructor) \bowtie teaches) \bowtie course)$$

Este plan puede describirse así:

1. Filtrar **instructor** para conservar solo los instructores del departamento **Music**.
2. Unir ese resultado con **teaches**.
3. Unir el resultado con **course**.
4. Proyectar los atributos **name** y **title**.

Este plan suele ser más eficiente porque reduce la cantidad de tuplas antes de realizar los joins.

7 Estadísticas supuestas

Supongamos las siguientes estadísticas de las relaciones:

Relación o dato	Cantidad aproximada
instructor	1000 filas
teaches	10000 filas
course	500 filas
Instructores del departamento Music	20 filas
Cursos dictados en promedio por instructor	4 cursos

Estas estadísticas permiten estimar el costo relativo de los planes.

8 Estimación del Plan A

En el Plan A, primero se realiza el join:

$$instructor \bowtie teaches$$

Como **teaches** tiene aproximadamente 10000 filas, el resultado intermedio puede tener aproximadamente:

10000 filas

Luego se une ese resultado con **course**:

$$(instructor \bowtie teaches) \bowtie course$$

El resultado sigue teniendo aproximadamente:

10000 filas

Después se aplica la selección:

$$\sigma_{dept_name='Music'}$$

Si solo 20 de los 1000 instructores pertenecen al departamento **Music**, la proporción de instructores de ese departamento es:

$$\frac{20}{1000} = 0.02$$

Entonces, el resultado estimado luego del filtro es:

$$10000 \times 0.02 = 200 \text{ filas}$$

El resultado final tiene aproximadamente 200 filas, pero antes se procesaron resultados intermedios de unas 10000 filas.

9 Estimación del Plan B

En el Plan B, primero se aplica la selección:

$$\sigma_{dept_name='Music'}(instructor)$$

El resultado contiene aproximadamente:

$$20 \text{ filas}$$

Luego se realiza el join con **teaches**:

$$\sigma_{dept_name='Music'}(instructor) \bowtie teaches$$

Si cada instructor dicta en promedio 4 cursos, el resultado estimado es:

$$20 \times 4 = 80 \text{ filas}$$

Luego se une con **course**:

$$(\sigma_{dept_name='Music'}(instructor) \bowtie teaches) \bowtie course$$

El resultado intermedio se mantiene aproximadamente en:

$$80 \text{ filas}$$

Por lo tanto, este plan procesa muchos menos datos intermedios que el Plan A.

10 Comparación de los planes

Plan	Estrategia	Tamaño intermedio aproximado
Plan A	Join primero, filtro después	10000 filas
Plan B	Filtro primero, join después	80 filas

El optimizador probablemente elegiría el Plan B, porque su costo estimado es menor.

11 SQL que representa el Plan B

El Plan B puede expresarse explícitamente en SQL de la siguiente manera:

```
SELECT i.name, c.title
FROM (
    SELECT *
    FROM instructor
    WHERE dept_name = 'Music'
) i
JOIN teaches t
  ON i.ID = t.ID
JOIN course c
  ON t.course_id = c.course_id;
```

Esta consulta es equivalente a la consulta SQL inicial. La diferencia es que aquí se expresa explícitamente la idea de filtrar primero la relación `instructor`.

12 Conclusión

El criterio principal para elegir un plan de evaluación es el menor costo estimado.

En este ejemplo, ambos planes producen el mismo resultado:

$$\Pi_{name,title} (\sigma_{dept_name='Music'} (instructor \bowtie teaches \bowtie course))$$

Sin embargo, el Plan B es más conveniente porque aplica la selección antes de los joins:

$$\Pi_{name,title} ((\sigma_{dept_name='Music'} (instructor) \bowtie teaches) \bowtie course)$$

La idea central es:

filtrar temprano $\Rightarrow$ menos filas intermedias $\Rightarrow$ menor costo estimado

13 Ejemplo 2: optimización de una consulta

Considere las siguientes relaciones:

```
instructor(ID, name, dept_name, salary)
teaches(ID, course_id, sec_id, semester, year)
course(course_id, title, dept_name, credits)
```

Se desea obtener el nombre de los instructores del departamento **Music** que enseñaron cursos en el año 2017, junto con el título del curso.

13.1 Consulta SQL inicial

Dada la siguiente consulta:

```
SELECT i.name, c.title
FROM instructor i
JOIN teaches t
  ON i.ID = t.ID
JOIN course c
  ON t.course_id = c.course_id
WHERE i.dept_name = 'Music'
AND t.year = 2017;
```

Responder:

1. Escribir la expresión inicial en álgebra relacional.
2. Dibujar el árbol inicial.
3. Aplicar reglas de equivalencia para obtener una expresión más eficiente.
4. Dibujar el árbol optimizado.
5. Explicar por qué el segundo árbol puede ser más eficiente.
6. Comparar dos planes usando estadísticas supuestas.

14 Resultado esperado

14.1 Expresión inicial en álgebra relacional

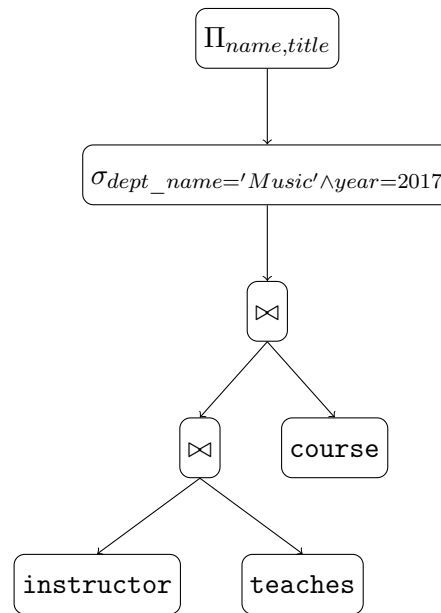
Una forma inicial de representar la consulta es:

$$\Pi_{name, title} (\sigma_{dept_name='Music' \wedge year=2017} ((instructor \bowtie teaches) \bowtie course))$$

Aquí:

- $\Pi_{name, title}$ representa la proyección final.
- $\sigma_{dept_name='Music' \wedge year=2017}$ representa las condiciones del **WHERE**.
- $\bowtie$ representa los joins entre las tablas.

14.2 Árbol inicial



Este árbol indica que primero se hacen los joins y recién después se aplican los filtros.

14.3 Aplicación de reglas de equivalencia

14.3.1 Paso 1: separar la selección compuesta

Se aplica la regla:

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

En este caso:

$$\theta_1 = dept_name = 'Music'$$

$$\theta_2 = year = 2017$$

Por lo tanto:

$$\sigma_{dept_name='Music' \wedge year=2017}(E)$$

se transforma en:

$$\sigma_{dept_name='Music'}(\sigma_{year=2017}(E))$$

14.3.2 Paso 2: empujar la selección sobre instructor

La condición:

$$dept_name = 'Music'$$

usa solamente atributos de la relación **instructor**.

Se aplica la regla:

$$\sigma_{\theta}(E_1 \bowtie E_2) \equiv \sigma_{\theta}(E_1) \bowtie E_2$$

si θ usa solo atributos de E_1 .

Entonces:

$$\sigma_{dept_name='Music'}(instructor \bowtie teaches)$$

se transforma en:

$$\sigma_{dept_name='Music'}(instructor) \bowtie teaches$$

14.3.3 Paso 3: empujar la selección sobre teaches

La condición:

$$year = 2017$$

usa solamente atributos de la relación **teaches**.

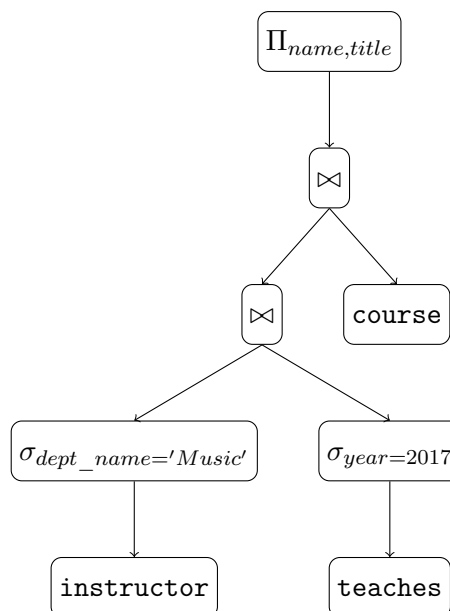
Por eso, la selección puede aplicarse antes del join:

$$\sigma_{year=2017}(teaches)$$

La expresión optimizada queda:

$$\Pi_{name,title} ((\sigma_{dept_name='Music'}(instructor) \bowtie \sigma_{year=2017}(teaches)) \bowtie course)$$

14.4 Árbol optimizado



En este árbol, los filtros se aplican antes de los joins.

14.5 SQL equivalente al árbol optimizado

```
SELECT i.name, c.title
```

```

FROM (
    SELECT *
    FROM instructor
    WHERE dept_name = 'Music'
) i
JOIN (
    SELECT *
    FROM teaches
    WHERE year = 2017
) t
ON i.ID = t.ID
JOIN course c
ON t.course_id = c.course_id;

```

Esta consulta es equivalente a la consulta original, pero muestra explícitamente que primero se filtran las tablas.

14.6 Comparación de planes

Supongamos las siguientes estadísticas:

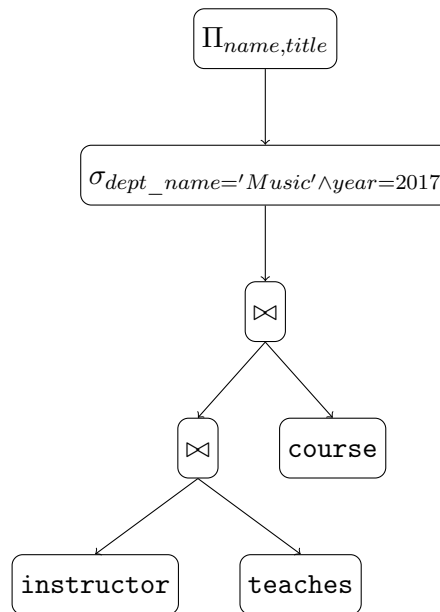
Relación o dato	Cantidad
<code>instructor</code>	1000 filas
<code>teaches</code>	10000 filas
<code>course</code>	500 filas
Instructores de <code>Music</code>	20 filas
Registros de <code>teaches</code> en 2017	2000 filas
Cursos promedio por instructor de <code>Music</code> en 2017	3 cursos

14.6.1 Plan A: joins primero, filtros después

Álgebra relacional:

$$\Pi_{name, title} (\sigma_{dept_name='Music' \wedge year=2017} ((instructor \bowtie teaches) \bowtie course))$$

Árbol del Plan A:



Estimación:

Primero se realiza:

$$instructor \bowtie teaches$$

Como **teaches** tiene aproximadamente 10000 filas, el resultado puede tener aproximadamente:

10000 filas

Luego:

$$(instructor \bowtie teaches) \bowtie course$$

sigue teniendo aproximadamente:

10000 filas

Después se aplican los filtros:

$$dept_name = 'Music'$$

y

$$year = 2017$$

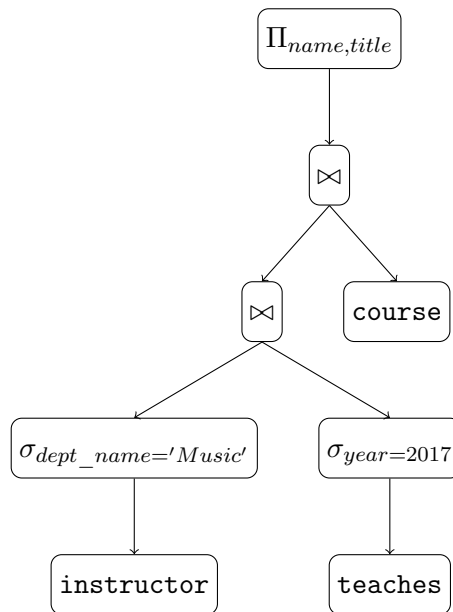
Aunque el resultado final sea pequeño, el plan procesó antes un resultado intermedio grande.

14.6.2 Plan B: filtros primero, joins después

Álgebra relacional:

$$\Pi_{name,title} \left(\left(\sigma_{dept_name='Music'}(instructor) \bowtie \sigma_{year=2017}(teaches) \right) \bowtie course \right)$$

Árbol del Plan B:



Estimación:

Primero se filtra **instructor**:

$$\sigma_{dept_name='Music'}(instructor)$$

Resultado estimado:

20 filas

Luego se filtra **teaches**:

$$\sigma_{year=2017}(teaches)$$

Resultado estimado:

2000 filas

Después se hace el join entre instructores de **Music** y registros de cursos dictados en 2017.

Si cada instructor de **Music** dictó aproximadamente 3 cursos en 2017:

$$20 \times 3 = 60 \text{ filas}$$

Luego se une con **course**, manteniendo aproximadamente:

60 filas

14.7 Comparación final

Plan	Estrategia	Tamaño intermedio aproximado
Plan A	Hacer joins y luego filtrar	10000 filas
Plan B	Filtrar primero y luego hacer joins	60 filas

El optimizador probablemente elegiría el Plan B, porque reduce mucho el tamaño de los resultados intermedios.

14.8 Conclusión

Este ejercicio muestra tres ideas centrales de la optimización de consultas:

1. Una misma consulta puede tener varias expresiones equivalentes.
2. Las reglas de equivalencia permiten transformar una expresión en otra.
3. El optimizador elige el plan con menor costo estimado, generalmente intentando reducir cuanto antes el tamaño de los resultados intermedios.

La idea principal es:

filtrar temprano $\Rightarrow$ menos tuplas intermedias $\Rightarrow$ menor costo estimado